

Reviewer Pack — Minimal Index of Siegel Modular Forms and Communication Comp...

Entry 1b941866984f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-02 18:43:33 UTC

Minimal Index of Siegel Modular Forms and Communication Complexity Rank Correlation

Entry ID: 1b941866984f **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-02 18:43:33 UTC

1. Conjecture

Field A (mathematical branch): Siegel Modular Form Theory **Field B** (complexity object): Communication Complexity

Statement:

For every integer $N \geq 2$, let f_N be a random Boolean function of n variables with communication complexity rank $r(f_N)$. The minimal index of the Siegel modular form associated with f_N is bounded by $O(r(f_N) \log n)$, i.e., $\min_index(f_N) = \Theta(r(f_N) \log n)$.

Rationale (proposer's reasoning):

Siegel modular forms provide a rich algebraic structure that could potentially encode non-trivial information about computational problems, such as communication complexity. If this conjecture holds, it would suggest a novel connection between number theory and complexity theory, potentially leading to new algorithms or insights into the structure of Boolean functions.

Taxonomy category: Siegel Modular Form Theory $\times$ Communication Complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 162eb5b01e5ba0b2

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

To support the conjecture, the mean of the ratio of minimal index to communication complexity rank over all generated Boolean functions must be ≤ 3 times $\log n$ with a confidence interval that does not cross this threshold.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Siegel modular form" AND communication complexity" - "minimal index" AND "communication complexity rank" - "Boolean function" AND Siegel modular forms AND communication complexity

Top relevant hits considered: - [http://arxiv.org/abs/2109.11753v1] Pullback formula for vector valued Siegel modular forms and its applications - [http://arxiv.org/abs/2305.00836v1] On the existence of Siegel modular forms with extra twists - [http://arxiv.org/abs/2502.06841v1] Constructing Arithmetic Siegel Modular Forms: Theta Lifting and Explicit Methods for Real Multiplication Abelian Surface - [http://arxiv.org/abs/1501.03444v1] Average case complexity of DNFs and Shannon semi-effect for narrow subclasses of boolean functions - [http://arxiv.org/abs/0909.3392v2] On the communication complexity of XOR functions - [http://arxiv.org/abs/1005.2254v9] On Universal Complexity Measures

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def communication_complexity_rank(f):
        n = int(math.log2(len(f)))
        rank = 0
        for i in range(2**n):
            if f[i] == 1:
                rank += 1
        return rank

    def minimal_index(f, n):
        A = []
        b = []
        for i in range(2**n):
            row = [f[j] ^ (i >> j & 1) for j in range(n)]
            A.append(row)
            b.append(i % 2)
```

```

# Gaussian elimination
m, k = len(A), n
for i in range(m):
    if A[i][i] == 0:
        for j in range(i + 1, m):
            if A[j][i] != 0:
                A[i], A[j] = A[j], A[i]
                break
        else:
            return None # Singular matrix

    for j in range(i + 1, m):
        factor = Fraction(A[j][i], A[i][i])
        for k in range(k):
            A[j][k] -= factor * A[i][k]

# Count non-zero rows
min_index_val = sum(1 for row in A if any(row))
return min_index_val

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    f = generate_random_boolean_function(n)
    rank = communication_complexity_rank(f)
    if rank is None:
        continue

    min_index_val = minimal_index(f, n)
    if min_index_val is None:
        continue

    results.append({
        "n": n,
        "rank": rank,
        "min_index": min_index_val
    })

if not results:
    return {
        "metric_name": "minimal_index_to_rank_ratio",
        "metric_value": 0.0,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

mean_ratio = sum(result["min_index"] / result["rank"] for result in results) / len(results)
std_ratio = math.sqrt(sum((result["min_index"] / result["rank"] - mean_ratio) ** 2 for result

return {
    "metric_name": "minimal_index_to_rank_ratio",

```

```

        "metric_value": mean_ratio,
        "instances_tested": len(results),
        "n_max": max(result["n"] for result in results),
        "conjecture_holds": mean_ratio <= 3 * math.log(max(result["n"] for result in results)),
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_ratio = sum(r["metric_value"] for r in results) / len(results)
    std_ratio = math.sqrt(sum((r["metric_value"] - mean_ratio) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_ratio} std={std_ratio} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_ratio} std={std_ratio} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f3309870.py", line 108, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f3309870.py", line 70, in run_trial
    min_index_val = minimal_index(f, n)
                    ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f3309870.py", line 43, in minimal_index
    if A[i][i] == 0:
        ~~~^
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means it did not complete its execution to provide a result for the conjecture. | next: Re-run the test with appropriate error handling and ensure that it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	32908
2	propose	ollama_remote	glm4:latest	0	0	23757
3	preregistration	ollama_remote	glm4:latest	0	0	18236
4	novelty	ollama_remote	glm4:latest	0	0	8454
5	novelty	ollama_remote	glm4:latest	0	0	13860
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15156
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17668
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11257
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18281
10	judge	ollama_remote	glm4:latest	0	0	11324

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 170901 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/1b941866984f.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/1b941866984f.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/1b941866984f.tar.gz (if generated)